

NFL Prediction System — Product Requirements Document (PRD)

Owner: Jack

Version: v1.0

Date: 2025-09-08 (America/New_York)

1) Summary

A Python-based system that produces **pre-game predictions** for every NFL matchup each week: - **Win Probability (WP)** for home/away.

- **Against the Spread (ATS)**: predicted cover probability and fair spread.

- **Totals (O/U)**: predicted game points distribution, fair total, and over/under probabilities.

The system includes:

(a) a reproducible **data pipeline** (games, odds, weather, team performance metrics),

(b) a modeling stack (Elo baseline → ML models with calibration),

(c) a **walk-forward backtest** with strict no-leakage rules and betting viability metrics, and

(d) a **simple web UI** to browse weekly games, model outputs, and edges.

2) Goals & Non-Goals

Goals (v1)

1. Generate calibrated **pre-game probabilities** for WP, ATS, and O/U by a fixed **snapshot time** (default: **Friday 6:00 PM ET**).
2. End-to-end reproducibility: deterministic data snapshots, artifact versioning, and config-driven runs.
3. Transparent evaluation: walk-forward backtest by season (≥ 5 seasons), metrics (LogLoss/Brier; MAE/RMSE), and simulated betting PnL + CLV.
4. Lightweight UI to display the current week's slate, predictions, and recommended bets given an edge threshold.

Non-Goals (v1)

- Live in-game modeling.
 - Paid/proprietary injury feeds or player-level tracking data.
 - Arbitrage execution or sportsbook account management.
-

3) Users & Primary Use Cases

- **Researcher/Engineer (you):** iterate on features/models, inspect calibration, and validate edges.
- **Power user:** browse weekly predictions and recommended bets.

Use cases: - "Show Week 6 predictions as of Friday 6pm ET."

- "Which games show $\geq 2.5\%$ edge vs. market?"

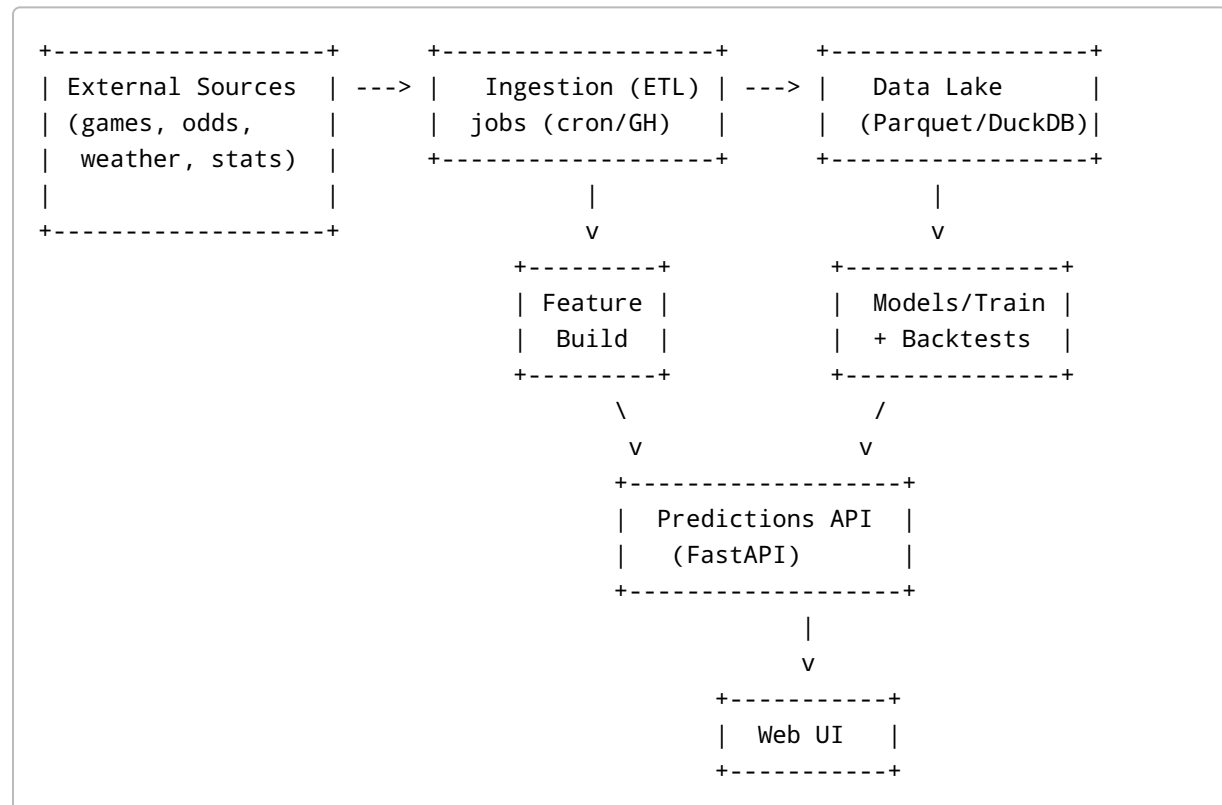
- "How did the model perform last season and YTD?"

4) Scope (v1 $\rightarrow$ v1.1)

v1 (this PRD): Elo baseline + logistic regression (WP), gradient boosting (ATS/Total), isotonic calibration, weekly pipeline, UI table.

v1.1 (fast follow): offense/defense split ratings, QB-adjusted ratings, feature importance views, cohort analysis (wind/outdoors).

5) System Architecture Overview



6) Data Sources (free/low-friction)

Note: abstract behind a `DataProvider` interface to swap vendors if needed.

- **Games & Team Performance:** season schedule, historical results, team-level box/pbp aggregates. Use `nfl_data_py` (which mirrors widely used public CSVs).
- **Odds:** opening/current moneyline, spread, total (snapshot at run time). Default: TheOddsAPI-style JSON feed (requires API key).
- **Weather:** forecast by stadium lat/lon for game kickoff (temperature, wind, precip). Default: Meteostat JSON/Python.
- **Stadium/Meta:** venue type (indoor/outdoor/retractable), home team, time zone, coordinates (static JSON mapping in repo).

Optional (v1.1+): injury reports (manual scrapes), QB starter probabilities, pressure allowed/created (if you add a source).

7) Data Ingestion & Storage

7.1 Schedules & Results

- Pull season schedules and results by week.
- Normalize team names (canonical `team_id`).
- Persist **raw pulls** under `data/raw/date=YYYY-MM-DD/` as JSON/CSV.

7.2 Odds Snapshots

- For each upcoming game: capture **snapshot** of odds at **Friday 6pm ET** and store under `data/raw/odds/date=YYYY-MM-DD/`.
- Fields: sportsbook, timestamp, moneyline_home/away, spread, total, juice.

7.3 Weather

- Query forecast for each outdoor/retractable stadium at kickoff time (rounded to hour).
- Fields: temp_f, wind_mph, precip_prob, condition_code.

7.4 Storage Layer

- **File format:** Parquet for columnar analytics.
 - **Query engine:** DuckDB for local analytics, Pandas for modeling.
 - **Data model:** “bronze/silver/gold” folders:
 - `bronze/` raw snapshots
 - `silver/` cleaned tables (games, odds, weather, team_stats)
 - `gold/` feature matrices per target (wp, ats, ou)
-

8) Data Schemas (key tables)

8.1 `games` (silver)

column	type	notes
game_id	string	season_week_teamkey (e.g., 2024_W06_KC@BUF)
season	int	NFL year
week	int	1–18
kickoff_et	datetime	ET zone
home_team	string	canonical ID
away_team	string	canonical ID
venue	string	name
venue_roof	enum	indoor/outdoor/retractable
home_score	int	final (historical)
away_score	int	final (historical)
result	int	+1 home win, 0 loss/tie (for training)

8.2 `odds_snapshot` (silver)

column	type	notes
game_id	string	FK → games
snapshot_ts	datetime	Friday 18:00:00 ET
sportsbook	string	id
ml_home	int	moneyline
ml_away	int	moneyline
spread	float	home spread (-3.5)
spread_ju_home	int	e.g., -110
spread_ju_away	int	e.g., -110
total	float	e.g., 47.5
total_over_ju	int	
total_under_ju	int	

8.3 weather_forecast (silver)

column	type	notes
game_id	string	FK
temp_f	float	
wind_mph	float	
precip_prob	float	0–1
is_outdoor	bool	derived from venue_roof

8.4 team_form (silver)

Rolling aggregates up to the snapshot: | column | type | notes | |---|---|---| | team_id | string | | | season | int | | | week | int | "as of" week | | epa_off_l4 | float | rolling 4-week EPA/play offense | | epa_def_l4 | float | rolling 4-week EPA/play defense | | succ_off_l4 | float | success rate offense | | succ_def_l4 | float | success rate defense | | neutral_pass_rate_l4 | float | 1st/2nd down, win prob 20–80% | | rest_days | int | since last game |

8.5 features_* (gold)

Separate matrices for wp, ats, ou, each with: - X columns: Elo diff, home flag, rest_days, travel_tz_diff, venue_roof flags, weather (wind/temp), rolling EPA/success/neutral pass, market anchors (opening/current).

- y targets: - wp: home_win (0/1)
- ats: home_margin – spread (or cover indicator)
- ou: total_points (regression) and/or Over indicator relative to market

9) Feature Engineering (v1)

- **Ratings:** Elo (team-level) with margin-of-victory and dynamic K; home-field advantage learned per season.
- **Form:** rolling 4-week EPA/play and success rates (off/def).
- **Tendencies:** neutral-situation pass rate or pass-rate-over-league-expectation approximation.
- **Context:** rest days, short week, travel time-zone difference, venue roof.
- **Weather:** wind (primary), temp, precip, outdoor flag.
- **Market Anchors:** opening line and current line at snapshot (never closing).
- **Regularization:** winsorize outliers; z-score within season.

Leakage guardrails:

- Build form metrics using only games strictly **before** the prediction week.
- For the current week, do not include any opponent post-game stats.
- Use odds only from the **snapshot time**.

10) Modeling

10.1 Targets

- **WP**: probability home team wins (away WP = $1 - \text{home WP}$).
- **ATS**:
 - Regression: predict **expected margin**; derive cover probability vs market spread (assume residuals $\sim \text{Normal}(\sigma_{\text{ats}})$).
 - Or classification: cover indicator; output calibrated probability.
- **Totals (O/U)**: predict **expected total points**; assume residuals $\sim \text{Normal}(\sigma_{\text{ou}})$ to derive over/under probabilities at market total.

10.2 Algorithms (v1)

- **Baseline**: Elo $\rightarrow$ logistic for WP.
- **WP model**: Logistic Regression (`liblinear` / `lbfgs`) on Elo diff + key features.
- **ATS & O/U**: Gradient Boosted Trees (XGBoost or LightGBM) for regression; optionally Poisson-mixture baseline for points.
- **Calibration**: Isotonic Regression (preferred) or Platt scaling on validation folds **within each season** (or nested out-of-time CV).
- **Uncertainty**: Estimate residual σ via out-of-time errors per season; use for probability transforms.

10.3 Training Protocol

- **Walk-forward**: For season Y, train on $\leq Y-1$, validate on Y (week-by-week).
- **Hyperparams**: coarse grid; prioritize stability over tiny gains.
- **Feature importance**: gain/SHAP (only for diagnostics; do **not** overfit to it).

11) Backtesting & Evaluation

11.1 Metrics

- **WP**: LogLoss, Brier, Calibration curve (ECE), Reliability diagram.
- **ATS/O-U regression**: MAE, RMSE; implied cover/over probabilities vs market, LogLoss for derived classifications.
- **Betting viability**: Simulated ROI at -110 (break-even 52.38%), **edge buckets**, and **Closing Line Value (CLV)** if closing data is available.

11.2 Rules

- Time-split only; never random CV.
- Lock data to snapshot time.
- Report season-by-season and aggregate stats (mean, std, CI via bootstrap).
- Sensitivity cohorts: outdoors vs indoors; wind ≥ 12 mph; short week; cross-time-zone travel.

12) Recommendations/Bet Sizing (Optional)

- **Edge definition (WP):** $\text{model_prob} - \text{market_implied_prob}$ (moneyline).
 - **Edge thresholds:** default $\geq 2.0\%$ to surface; configurable.
 - **Kelly (fractional):** optional $0.25\times$ Kelly based on edge; cap max stake.
 - **CLV tracking:** compare your fair line vs closing market (if collected).
-

13) Web UI (simple)

13.1 Tech

- **Backend:** FastAPI serving JSON + server-rendered Jinja templates.
- **Frontend:** Minimal Tailwind CSS table + client-side sorting/filtering (Alpine.js or vanilla).

13.2 Pages

1. **/ (Current Week):**
2. Table of games: date/time ET, teams, venue/roof, market (ML, spread, total), model WP/ATS/OU probabilities, **edge** columns, recommended bets (if any).
3. Filters: week selector, edge threshold slider, outdoors toggle.
4. Visual cues: color bands for edges ($\pm$).
5. **/game/{game_id}:** detail view with model inputs (features), historical head-to-head summary, calibration mini-chart.
6. **/reports:** calibration curves, backtest metrics by season, edge bucket performance.

13.3 UX Notes

- Mobile-friendly (one-column stack).
 - Download CSV buttons for current predictions and backtest results.
 - "As of" timestamp prominently displayed.
-

14) API (FastAPI)

14.1 Endpoints

- `GET /api/weeks/current` → current season/week metadata.
- `GET /api/games?season=&week=` → list games + market + predictions.
- `GET /api/games/{game_id}` → single game with features & probabilities.
- `GET /api/reports/backtest` → summary metrics.
- `GET /api/reports/calibration?target=` → reliability data points.

14.2 Response Shapes (examples)

- **Game item:**

```
{
  "game_id": "2025_W06_KC@BUF",
  "kickoff_et": "2025-10-12T16:25:00",
  "home": "BUF",
  "away": "KC",
  "market": {"ml_home": -130, "ml_away": +115, "spread": -2.5, "total":
48.5},
  "model": {
    "wp_home": 0.57,
    "fair_ml_home": -133,
    "cover_prob_home": 0.54,
    "fair_spread": -3.1,
    "over_prob": 0.51,
    "fair_total": 49.3
  },
  "edges": {"ml_home": +0.012, "spread_home": +0.018, "over": -0.003},
  "as_of": "2025-10-10T18:00:00-04:00"
}
```

15) Configuration & Secrets

- `.env` (never commit): `ODDS_API_KEY=...`
- `config.yaml`:

```
snapshot_time_et: "Friday 18:00"
seasons_backtest: [2018, 2019, 2020, 2021, 2022, 2023, 2024]
edge_threshold: 0.02
models:
  wp:
    algo: "logreg"
    calibrate: true
  ats:
    algo: "xgboost_reg"
  ou:
    algo: "xgboost_reg"
paths:
  data_root: "./data"
  artifacts_root: "./artifacts"
```

16) Orchestration & Deployment

- **Runner:** cron (local or server) or GitHub Actions workflow.

- **Schedule:**
 - **Fri 5:00 PM ET:** update data (schedules, team form).
 - **Fri 6:00 PM ET:** odds+weather snapshot → features → train/update → predict → publish artifacts → refresh UI cache.
 - **Artifacts:** `outputs/preds_{season}_{week}.parquet` + JSON for API.
 - **Hosting:** Uvicorn/Gunicorn behind Nginx (or simple `uvicorn` for dev).
 - **Static:** ship Tailwind CSS precompiled.
-

17) Observability & Quality

- **Logging:** structlog/standard logging with request IDs.
 - **Data QA checks:**
 - Row counts per week; missing odds/weather; duplicate games.
 - Sanity rules (spreads in [-20, +20], totals in [30, 65]).
 - **Model QA:**
 - Drift monitor: season-over-season LogLoss deltas; weekly calibration ECE.
 - Alert on missing snapshot or zero predictions.
-

18) Testing Strategy

- **Unit tests:** feature builders, Elo updates, probability transforms, moneyline ↔ probability conversions.
 - **Integration tests:** end-to-end dry run on a past week.
 - **Backtest tests:** spot-check a known season vs. frozen artifacts.
 - **UI tests:** endpoint contract tests + snapshot tests for HTML table.
-

19) Security & Compliance

- Keep API keys in environment variables; never commit.
 - Read-only ingestion; no credentials stored in client.
 - Rate-limit external API calls; exponential backoff & retries.
-

20) Performance & Scalability

- Historical training fits comfortably on a laptop (DuckDB + Parquet).
 - Weekly runs O(minutes).
 - Cache predictions for UI; avoid recompute on every request.
-

21) Risks & Mitigations

- **API limits/outages:** local caching; fallback mirrors; exponential backoff.
 - **Data drift (rules changes, extra week):** version features; assert column contracts.
 - **Market efficiency:** treat edges as research; use conservative thresholds.
-

22) Acceptance Criteria (v1)

1. Command `make snapshot` produces silver/gold tables for the current week.
 2. Command `make backtest` runs walk-forward 2018–2024 and writes a metrics report (HTML) + CSV.
 3. Command `make predict` writes `outputs/preds_{season}_{week}.parquet` and JSON.
 4. Web UI at `/` lists all current week games with model outputs and edges.
 5. Calibration page shows reliability curve for WP and ATS.
-

23) Project Structure

```
project/
├─ data/
│   ├── raw/ (bronze)
│   ├── silver/
│   └── gold/
├─ scripts/
│   ├── ingest_games.py
│   ├── ingest_odds.py
│   ├── ingest_weather.py
│   └── build_features.py
├─ ratings/
│   └── elo.py
├─ models/
│   ├── train_wp.py
│   ├── train_ats.py
│   ├── train_ou.py
│   ├── calibrate.py
│   └── utils.py
├─ backtest/
│   ├── walkforward.py
│   └── metrics.py
├─ api/
│   ├── main.py (FastAPI)
│   └── schemas.py
└─ web/
```

```

|   ├── templates/
|   └── static/
├── conf/
|   └── config.yaml
├── tests/
├── outputs/
├── Makefile
├── pyproject.toml (or requirements.txt)
└── README.md

```

24) Dependencies (Python)

- **Core:** python>=3.11, pandas, numpy, pyarrow, duckdb, polars (optional)
- **Modeling:** scikit-learn, xgboost (or lightgbm), statsmodels
- **API/UI:** fastapi, uvicorn, jinja2, pydantic, python-multipart, itsdangerous
- **HTTP/ETL:** httpx, tenacity, pytz, python-dateutil, pendulum, tqdm, pydantic-settings
- **Data:** nfl_data_py (games/stats), meteostat (weather)
- **Viz (reports):** matplotlib, plotly (optional)
- **Testing:** pytest, pytest-cov, hypothesis (optional)

25) Implementation Notes

25.1 Elo (team-level)

- **Init rating:** 1500 baseline.
- **Home-field:** learn per season via logistic fit; apply as Elo points.
- **K factor:** base $K=20 \times \text{MoV multiplier}$ (e.g., $1 + |\text{margin}|/14$), cap at $\times 2$.
- **Update:** chronological across seasons; carryover shrink 25 Elo points between seasons.
- **Conversion:** $p_{\text{home}} = 1 / (1 + 10^{(-\text{elo_diff}/400)})$.

25.2 Market conversions

- Moneyline to prob: negative ML $-A \rightarrow A/(A+100)$; positive ML $+B \rightarrow 100/(B+100)$; de-vig via proportional normalization across sides.

25.3 Probability transforms

- For ATS/O-U regression outputs, derive probability using Normal CDF with season-level residual σ ; estimate σ from out-of-time residuals.

25.4 Calibration

- Use held-out folds within each season; train isotonic on validation and apply to test weeks of that season, or do nested rolling window.
-

26) Example CLI

```
# Ingest & build features for current week
python scripts/ingest_games.py --season 2025 --week current
python scripts/ingest_odds.py --snapshot "2025-10-10T18:00:00-04:00"
python scripts/ingest_weather.py --season 2025 --week current
python scripts/build_features.py --season 2025 --week current

# Train + predict (uses config.yaml)
python models/train_wp.py --season 2025 --week current
python models/train_ats.py --season 2025 --week current
python models/train_ou.py --season 2025 --week current

# Serve API/UI
uvicorn api.main:app --host 0.0.0.0 --port 8000
```

27) Reporting Artifacts

- `reports/backtest_{timestamp}.html` with:
 - Per-season LogLoss/Brier (WP), MAE/RMSE (ATS/O-U).
 - Calibration plots and ECE.
 - Edge bucket hit rates and ROI.
 - CLV histograms (if closing lines collected).
-

28) Timeline (suggested)

- **Week 1:** Data ingestion, Elo, feature scaffolding.
 - **Week 2:** WP logistic model + calibration; backtest harness.
 - **Week 3:** ATS/O-U regressors + probability transform; reporting.
 - **Week 4:** FastAPI + UI; polish; docs; acceptance criteria sign-off.
-

29) Glossary

- **EPA:** Expected Points Added.
- **WP:** Win Probability.

- **ATS:** Against the Spread.
 - **O/U:** Over/Under.
 - **CLV:** Closing Line Value.
 - **ECE:** Expected Calibration Error.
-

30) Legal & Ethics

- For research/entertainment; **not financial advice**.
 - Respect site/API terms of service; cache politely; attribute sources.
-

31) Future Work (beyond v1.1)

- QB-level ratings and injury-adjusted priors.
- Hierarchical Bayesian offense/defense ratings.
- Player-level features (pressures, target shares) if data source added.
- Live in-game win probability and totals updating.
- Automated model selection per cohort (outdoors, wind).